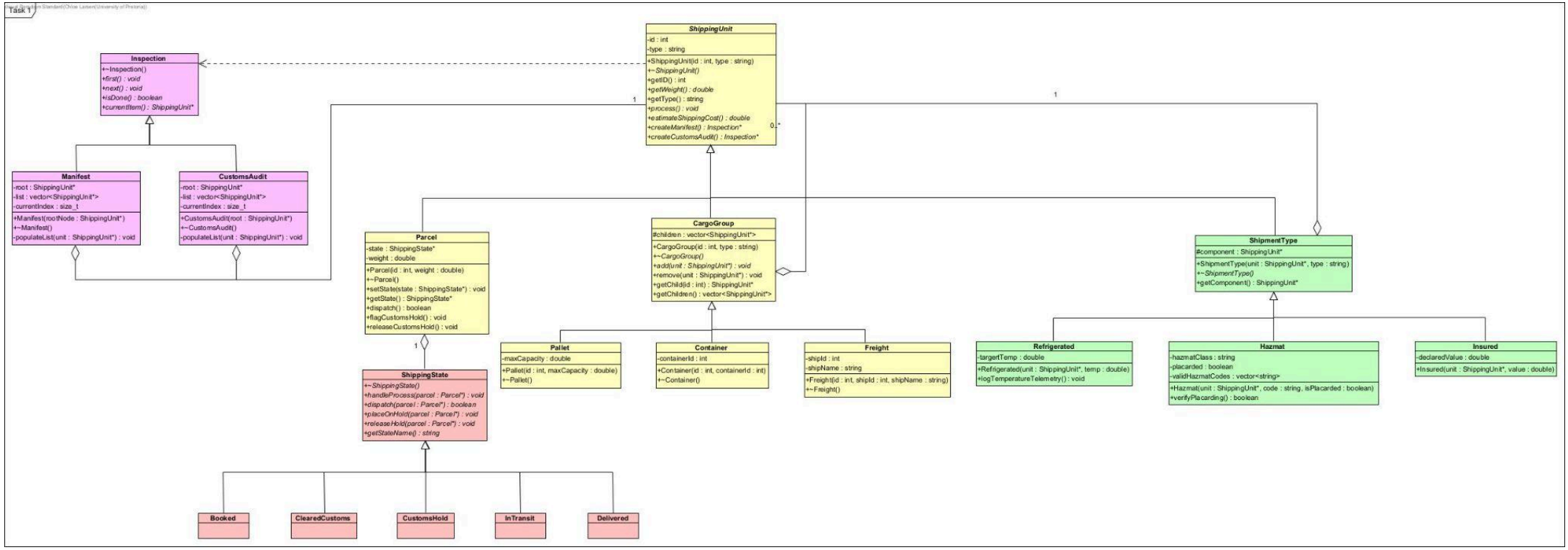
COS214 Practical Assignment 2

Jayden Dean - u24647897 Chloe Larsen - u25004141 Lindelwe Mabaleka - u25019385

Task 1



System Description:

TaskForge is an object-oriented shipping logistics application. It managed complex, multi-tiered freight structures(done with the composite pattern), with dynamic specifications (uses the decorated pattern, with parcels, containers, pallets or freights being wrapped by Hazmat - to ensure proper transport, Refrigerated - ensures item is stored at a specific temperature, Insured - ensures items in unit are protected in case of damage) and operational parcel lifecycles (completed with the State pattern, The cycle represented is Booked -> InTransit -> (CustomsHold -> ClearedCustoms) -> Delivered). With compliance auditing through the implementation of the Iterator pattern.

GOF Patterns and Participants

Composite

Component	ShippingUnit	Defines the common interface
Leaf	Parcel	Represents an individual shipping unit
Composite	CargoGroup, Container, Pallet, Freight	Keeps a collection of child ShippingUnit* objects and recursively defining operations

Decorator

Component	ShippingUnit	Represents a uniform target interface
ConcreteComponent	Parcel, CargoGroup, Container, Pallet, Freight	The core undecorated leaf entity that defines base implementation
Decorator	ShipmentType	Inherits from ShippingUnit and maintains a wrapped pointer of type ShippingUnit
ConcreteDecorator	Hazmat, Refrigerated, Insured	Adds specific items depending on the chosen decorator

State

Context	Parcel	Holds a pointer to a

		ShippingState *state and delegated dispatch, lifecycle logging, and customs
State	ShippingState	Declared polymorphic events like handleProcess(), dispatch(), placeOnHold(), releaseHold(), getStateName()
ConcreteState	Booked, InTransit, CustomsHold, ClearedCustoms, Delivered	Implements different behaviour associated with a state of the Context

Iterator

Iterator	Inspection	Defines the traversal methods
ConcreteIterator	Manifest, CustomsAudit	Implements the different traversal and population methods
Aggregate	ShippingUnit	Defines the factory methods createManifest() & createCustomsAudit()
ConcreteAggregate	Manifest, CustomsAudit	Implements the different traversal and population methods

Task 3

The program in `main.cpp` runs two connected runtime scenarios that exercise the design as a working system rather than a static demonstration.

Scenario 1 — Loading and dispatching: A `Freight` vessel is assembled as a composite of `Container`, `Pallet` and `Parcel` units, with several parcels wrapped in `Hazmat`, `Insured` and `Refrigerated` decorators (including a stacked `Refrigerated(Insured(...))`). The full manifest is traversed to report each unit's weight and cost, and parcels are then dispatched, driving `Booked → InTransit` state transitions.

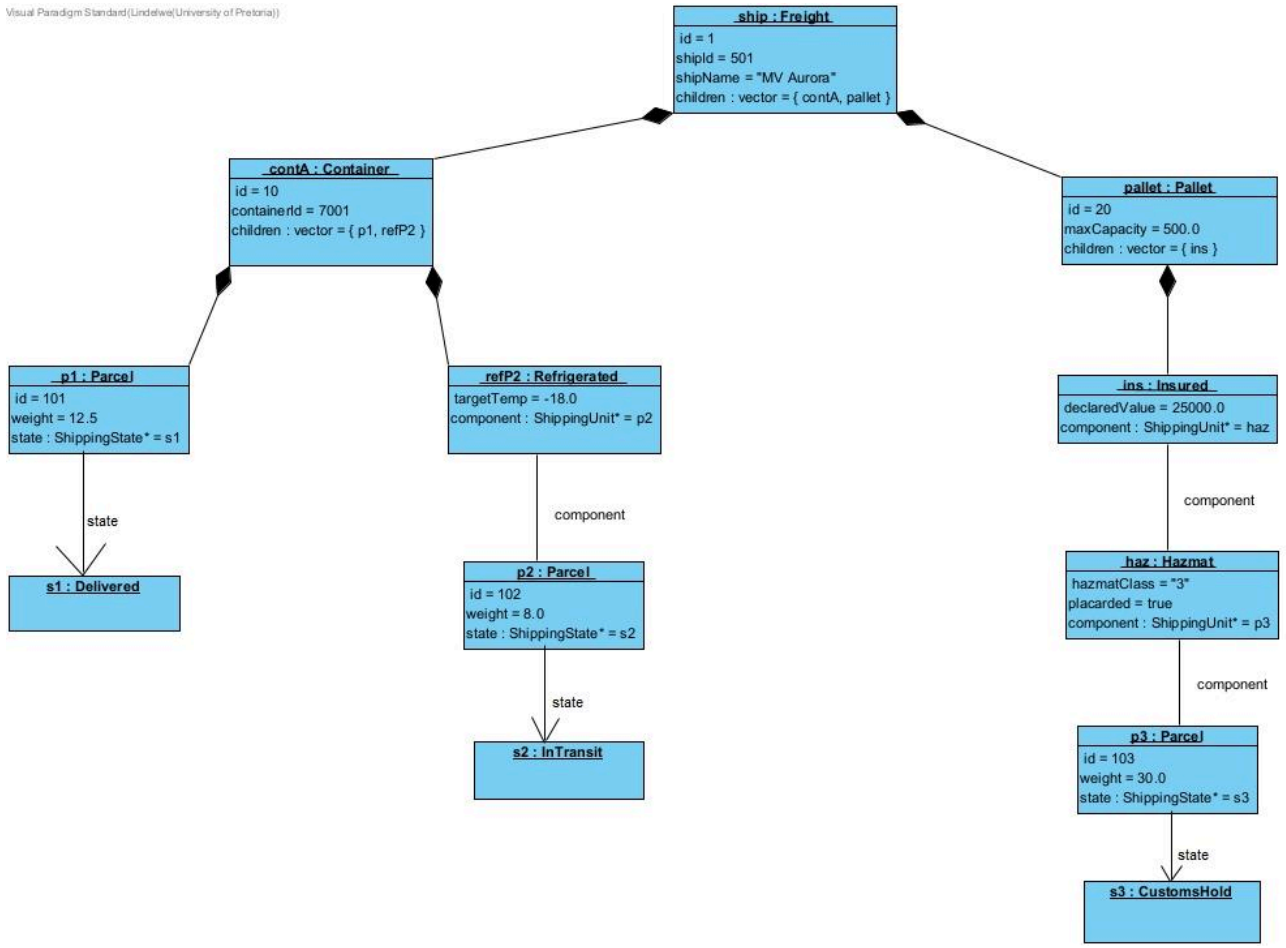
Scenario 2 — Customs inspection and structural change: A customs audit traverses the hierarchy and processes flagged units, triggering state-dependent behaviour (a Hazmat unit failing placarding is placed on `CustomsHold`; attempts to dispatch it while held are correctly rejected). The structure is then changed at runtime by offloading the non-compliant unit with `remove()`, after which the vessel's weight and cost are recomputed and a fresh audit

reflects the new structure. The hold is later released (`CustomsHold → ClearedCustoms → InTransit`) and remaining parcels are delivered.

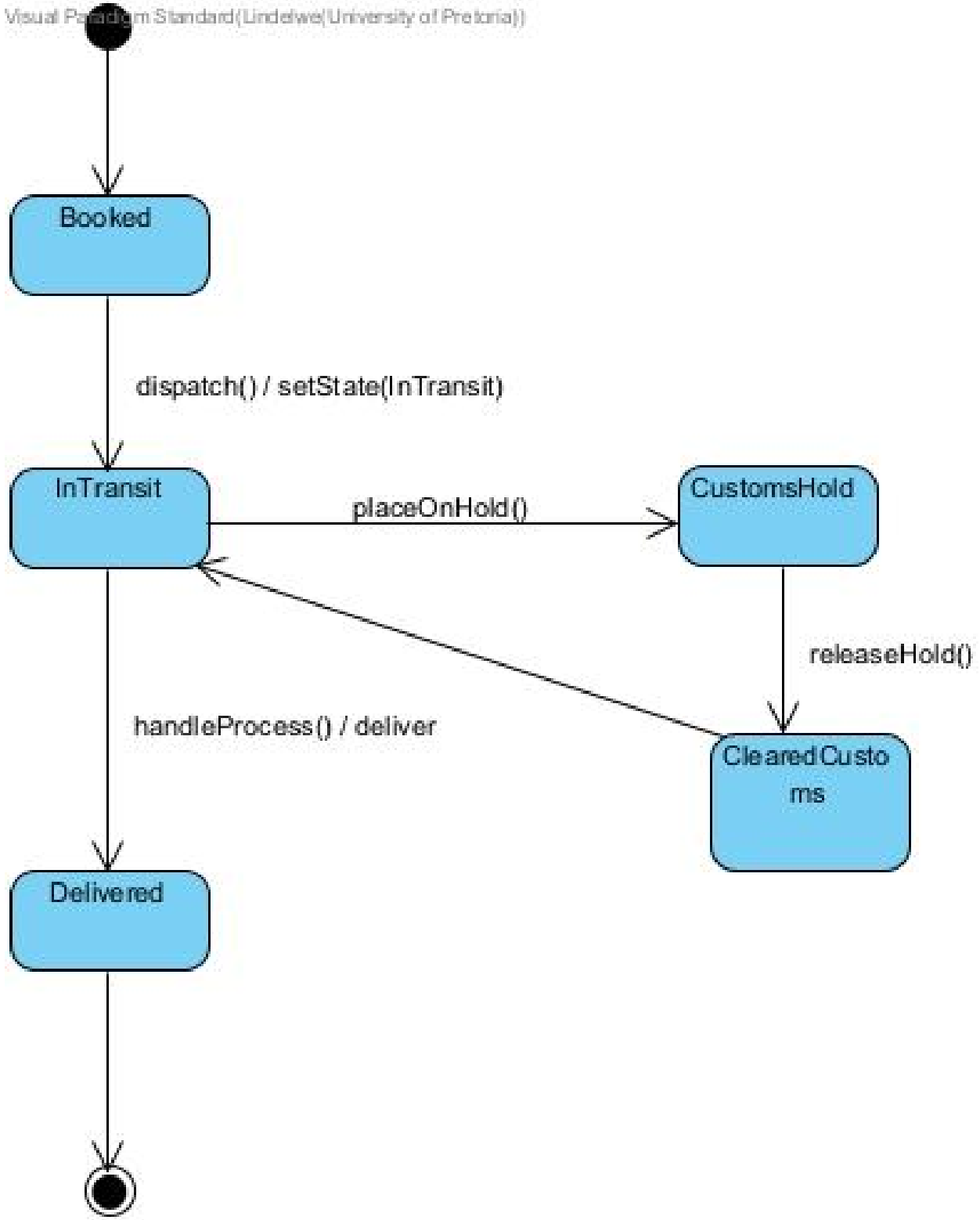
Traversal-modification policy: Our iterators re-materialise the tree when `first()` is called: each call clears the internal list and re-walks the current structure from the root. Consequently, a new traversal (or re-invoking `first()`) always reflects the live structure at that moment, and we never iterate over a stale cached view. When the structure changes mid-program, we create a fresh iterator to observe the updated hierarchy, keeping traversal behaviour predictable and consistent with the composite as it currently exists.

Task 4

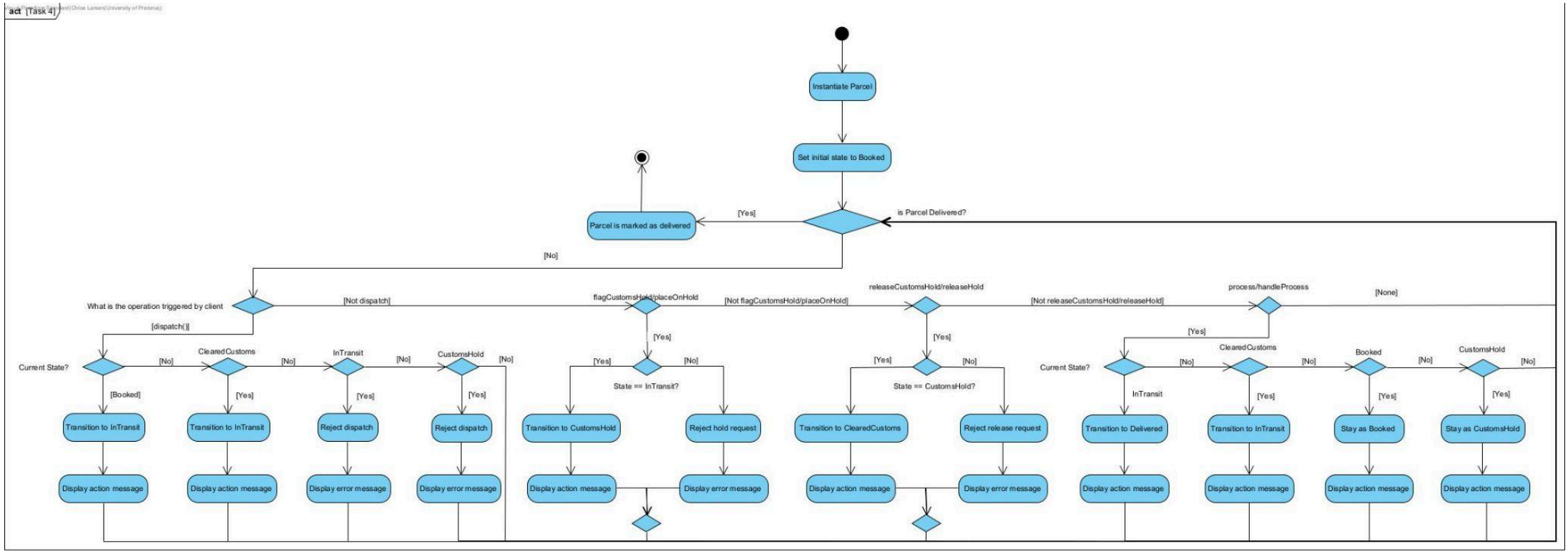
UML object diagram



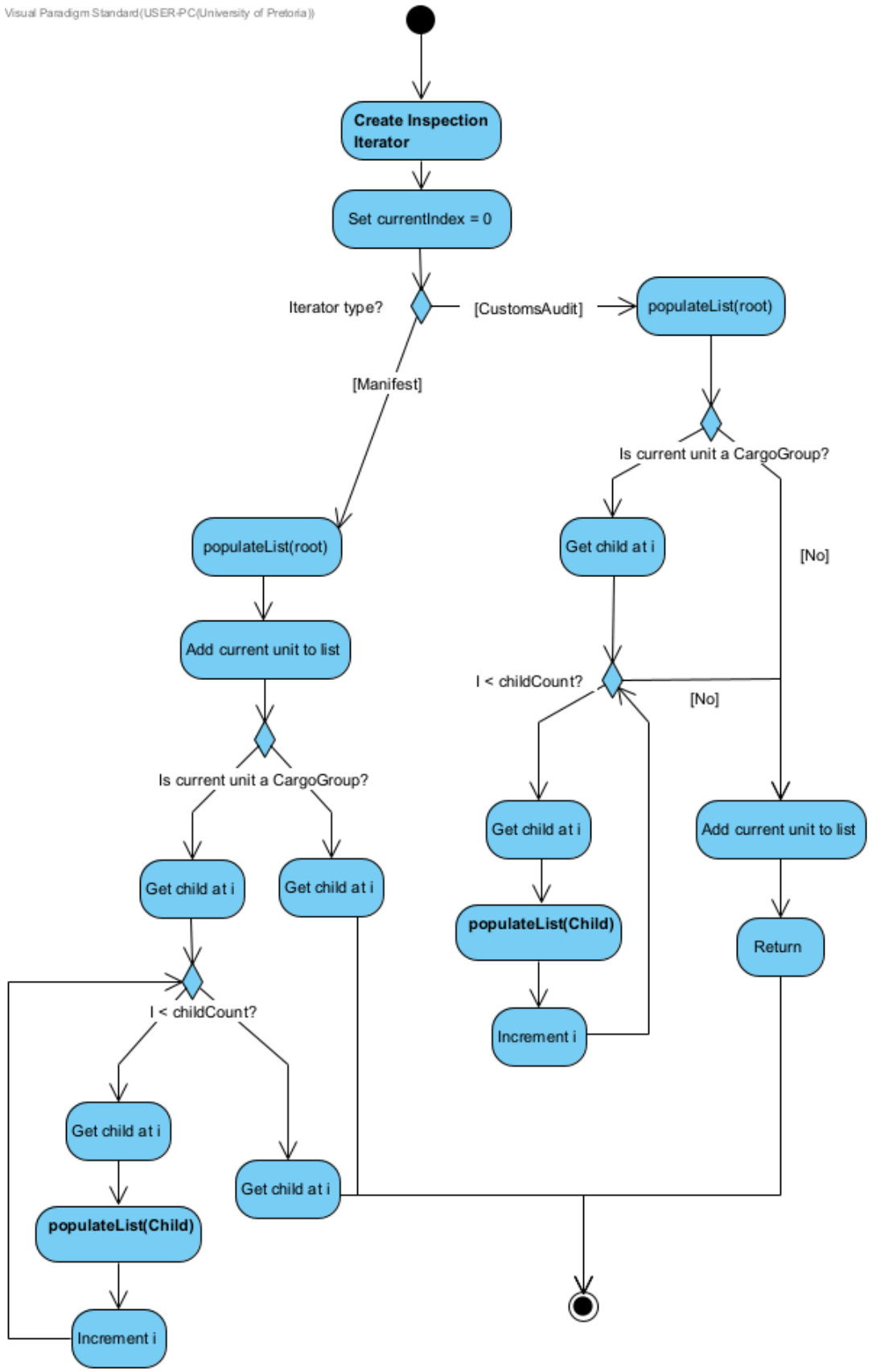
UML state diagram



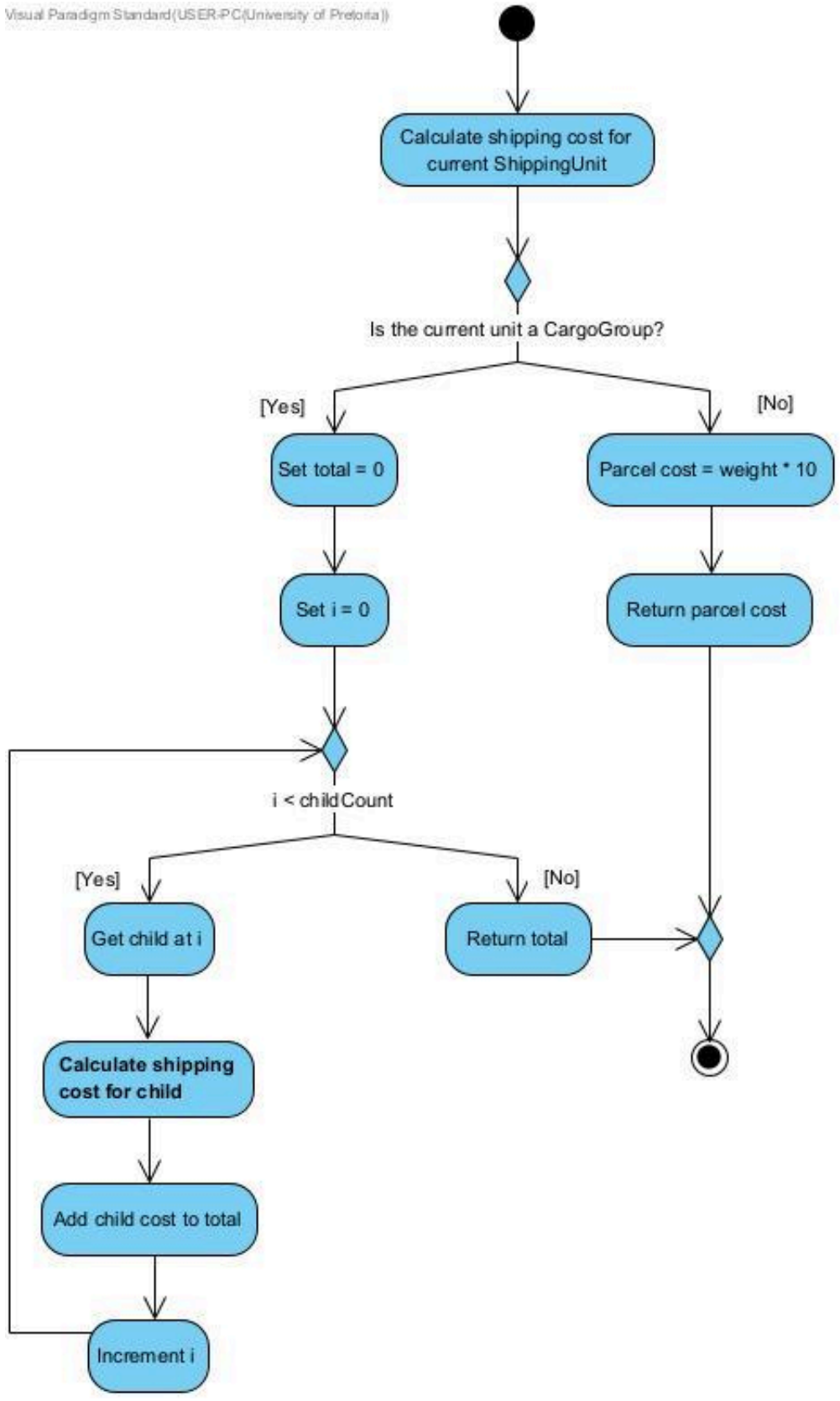
UML Activity Diagram - 1



UML Activity Diagram - 2



UML Activity Diagram - 3



Task 5

GDB

Using GDB:

```
Reading symbols from ./taskforge...
(gdb) break main.cpp:77
Breakpoint 1 at 0xe3ba: file main.cpp, line 77.
(gdb) break ShipmentType::estimateShippingCost
Breakpoint 2 at 0xd7b2: file src/ShipmentType.cpp, line 31.
(gdb) run
Starting program: /mnt/c/Users/chloe/OneDrive - University of
Pretoria/Second Year/COS 214/Practical/COS214-Prac4/taskforge

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to
.gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library
"/lib/x86_64-linux-gnu/libthread_db.so.1".

============================================================
        Creating composite tree
============================================================

Freight (#1) was created. The ships is called MSC Oscar with an id of
101.
Container (#10) with the containerId of 201 has been created.
Container (#11) with the containerId of 202 has been created.
Pallet (#100) has been created it has a max capacity of 1500kg
Pallet (#101) has been created it has a max capacity of 1200kg
Parcel (#1001) has been created. It has a weight of 20kg
Parcel (#1002) has been created. It has a weight of 35kg
Parcel (#1003) has been created. It has a weight of 15kg
Parcel (#1004) has been created. It has a weight of 50kg
Parcel(#1002) is now placarded to be of hazmat class of
UN1993-Flammable.
Parcel(#1003) is now placarded to be of hazmat class of
UN3480-LithiumBattery.
Parcel(#1004) is now insured with a declared value of R15000.
Parcel(#1004) is now refrigerated at a target temperature of -18°C.
                Parcel(id #1001) is now a part of Pallet(id #100)
```

```
                Parcel(id #1002) is now a part of Pallet(id #100)
                Parcel(id #1003) is now a part of Pallet(id #101)
                Parcel(id #1004) is now a part of Pallet(id #101)
                Pallet(id #100) is now a part of Container(id #10)
                Pallet(id #101) is now a part of Container(id #11)
                Container(id #10) is now a part of Freight(id #1)
                Container(id #11) is now a part of Freight(id #1)

Root Load: Freight (ID: #1)
Total Consignment Weight: 120 kg

Breakpoint 1, main () at main.cpp:77
77          std::cout << "Estimated Shipping Cost: R" <<
ColourHelper::BOLD << vessel->estimateShippingCost() <<
ColourHelper::RESET << std::endl;
(gdb) continue
Continuing.

Breakpoint 2, ShipmentType::estimateShippingCost (this=0x55555557da80)
at src/ShipmentType.cpp:31
warning: Source file is more recent than executable.
31          if (component)
(gdb) print component
$1 = (ShippingUnit *) 0x55555557d930
(gdb) print component->getType()
$2 = "Parcel"
(gdb) next
33              return component->estimateShippingCost();
(gdb) step
Parcel::getWeight (this=0x55555557d930) at src/Parcel.cpp:56
56          return weight;
(gdb) print weight
$3 = 35
(gdb) next
57      }
(gdb) finish
Run till exit from #0  Parcel::getWeight (this=0x55555557d930) at
src/Parcel.cpp:57
0x00005555555617de in ShipmentType::estimateShippingCost
(this=0x55555557da80) at src/ShipmentType.cpp:33
33              return component->estimateShippingCost();
Value returned is $4 = 35
```

Diagnosis:

- **Symptom:** During integration testing in main.cpp, calling vessel->estimateShippingCost() produced an unexpectedly low shipping quote.

Additionally, inspecting the decorator tier during cost evaluation returned numeric values that did not align with base shipping formula rates (weight * 2.50).

- **Cause:** In src/ShipmentType.cpp, the delegation method for cost estimation mistakenly returned component->getWeight() instead of invoking component->estimateShippingCost()

Valgrind

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes
./taskforge
==2506== Memcheck, a memory error detector
==2506== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2506== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright
info
==2506== Command: ./taskforge
==2506==

============================================================
        Creating composite tree
============================================================

Freight (#1) was created. The ships is called MSC Oscar with an id of
101.
Container (#10) with the containerId of 201 has been created.
Container (#11) with the containerId of 202 has been created.
Pallet (#100) has been created it has a max capacity of 1500kg
Pallet (#101) has been created it has a max capacity of 1200kg
Parcel (#1001) has been created. It has a weight of 20kg
Parcel (#1002) has been created. It has a weight of 35kg
Parcel (#1003) has been created. It has a weight of 15kg
Parcel (#1004) has been created. It has a weight of 50kg
Parcel(#1002) is now placarded to be of hazmat class of UN1993.
Parcel(#1003) is now placarded to be of hazmat class of UN3480.
Parcel(#1004) is now insured with a declared value of R15000.
Insured(#1004) is now refrigerated at a target temperature of -18°C.
                Parcel(id #1001) is now a part of Pallet(id #100)
                Hazmat(id #1002) is now a part of Pallet(id #100)
                Hazmat(id #1003) is now a part of Pallet(id #101)
                Refrigerated(id #1004) is now a part of Pallet(id #101)
                Pallet(id #100) is now a part of Container(id #10)
                Pallet(id #101) is now a part of Container(id #11)
                Container(id #10) is now a part of Freight(id #1)
                Container(id #11) is now a part of Freight(id #1)

Root Load: Freight (ID: #1)
Total Consignment Weight: 120 kg
```

```
Estimated Shipping Cost: R1950

============================================================
        Manifest Traversal (All Units)
============================================================

 [0] Freight    | ID: #1        | Weight: 120 kg        | Est. Cost:
R1950
 [1] Container  | ID: #10       | Weight: 55 kg | Est. Cost: R700
 [2] Pallet     | ID: #100      | Weight: 55 kg | Est. Cost: R700
 [3] Parcel     | ID: #1001     | Weight: 20 kg | Est. Cost: R200
 [4] Hazmat     | ID: #1002     | Weight: 35 kg | Est. Cost: R500
 [5] Parcel     | ID: #1002     | Weight: 35 kg | Est. Cost: R350
 [6] Container  | ID: #11       | Weight: 65 kg | Est. Cost: R1250
 [7] Pallet     | ID: #101      | Weight: 65 kg | Est. Cost: R1250
 [8] Hazmat     | ID: #1003     | Weight: 15 kg | Est. Cost: R300
 [9] Parcel     | ID: #1003     | Weight: 15 kg | Est. Cost: R150
 [10] Refrigerated      | ID: #1004     | Weight: 50 kg | Est. Cost:
R950
 [11] Insured   | ID: #1004     | Weight: 50 kg | Est. Cost: R875
 [12] Parcel    | ID: #1004     | Weight: 50 kg | Est. Cost: R500

============================================================
        State Transitions (Booked -> InTransit)
============================================================

State: Booked. -> InTransit
Dispatching unit(#1001) into transit.

State: Booked. -> InTransit
Dispatching unit(#1004) into transit.

State: InTransit
Unit (#1001) is already in transit.

State: Booked. -> InTransit
Dispatching unit(#1002) into transit.

State: Booked. -> InTransit
Dispatching unit(#1003) into transit.

============================================================
        Customs Audit Inspection
============================================================
```

```
[AUDITING UNIT] Hazmat (#1002)
Hazmat inspecting placarding for Parcel unit #1002 (Code: UN1993)
  --> REJECTED: Unit #1002 failed placarding verification! Flagging
customs hold.
State: InTransit  -> CustomsHold
Unit (#1002) was intercepted by border authorities. Placing on hold!
Parcel #1002 has a weight of 35kg
State: CustomsHold
Unit #1002 is locked in customs hold. Inspection required.

[AUDITING UNIT] Hazmat (#1003)
Hazmat inspecting placarding for Parcel unit #1003 (Code: UN3480)
  --> PASSED: Placard verified for UN Class UN3480.
Parcel #1003 has a weight of 15kg
State: InTransit  -> Delivered
Finalizing delivery for unit #1003.

============================================================
        Structural Change - Offloading Rejected Unit
============================================================

Offloading non-compliant Hazmat unit #1002 from Pallet #100:

Consignment metrics after offloading unit #1002:
Pallet #100 Updated Weight: 20 kg
Vessel Total Consignment Weight: 85 kg
Vessel Updated Shipping Cost: R1450

[AUDITING UNIT] Hazmat (#1003)
Hazmat inspecting placarding for Parcel unit #1003 (Code: UN3480)
  --> PASSED: Placard verified for UN Class UN3480.
Parcel #1003 has a weight of 15kg
State: Delivered
Unit #1003 has reached its destination. Lifecycle complete.

============================================================
        Resolving Holds & Finalising Delivery
============================================================

Attempting to dispatch held unit #1002 while on hold:
State: CustomsHold
Cannot dispatch unit #1002 currently on customs hold.

Releasing customs hold after placarding remediation:
State: CustomsHold  -> ClearedCustoms
Customs clearance granted for unit #1002. Hold released.
```

```
Resuming transit for unit #1002:
State: ClearedCustoms  -> InTransit
Dispatching unit #1002 back to transit.

Finalising deliveries at destination:
Parcel #1001 has a weight of 20kg
State: InTransit  -> Delivered
Finalizing delivery for unit #1001.

Parcel #1002 has a weight of 35kg
State: InTransit  -> Delivered
Finalizing delivery for unit #1002.

Parcel #1004 has a weight of 50kg
State: InTransit  -> Delivered
Finalizing delivery for unit #1004.
==2506==
==2506== HEAP SUMMARY:
==2506==     in use at exit: 0 bytes in 0 blocks
==2506==   total heap usage: 105 allocs, 105 frees, 78,703 bytes
allocated
==2506==
==2506== All heap blocks were freed -- no leaks are possible
==2506==
==2506== For lists of detected and suppressed errors, rerun with: -s
==2506== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 6

Workflow Reflection

Our team divided the practical using GitHub Issues, with each task assigned to a specific member so responsibilities were clear from the start. We worked on separate branches and integrated our work through pull requests, requiring at least one approving review before merging into `main`. This kept `main` stable and gave us a checkpoint to catch problems before they reached the shared codebase. Chloe set up the initial scaffold from the Task 1 class diagram, letting the rest of us work in parallel. Jayden authored the core model implementation, which Chloe integrated, refined for cleaner output, and fixed circular-dependency issues on. Chloe also wrote the runtime scenarios in `main.cpp`, the Makefile, and the Dockerfile. Lee produced the object and state diagrams and the README (PR #20) and reviewed PR #23. Coordinating over WhatsApp alongside GitHub let us flag blockers early — for example, the Task 3 scenarios depended on the core model, so we sequenced that work accordingly.

Team Contribution Statement

Lee (lee-codes012): Object diagram, state diagram, README (PR #20); review of PR #23; workflow reflection and contribution statement.
Chloe (Chloe-Larsen): Initial scaffold; runtime scenarios in main.cpp; Makefile and Dockerfile; integration and refinement of the core model (PR #23).
Jayden (Jaytad05): Core model implementation; README updates; activity diagrams.

https://github.com/Chloe-Larsen/COS214-Prac4